

Basic Structure of a SQL Query

This document outlines the fundamental components of a SQL query, providing clear explanations and practical examples for each clause. Understanding these building blocks is essential for effective data retrieval and manipulation.

Core SQL Query Clauses

A standard SQL **SELECT** statement typically includes several clauses that define what data to retrieve, from where, under what conditions, and how to organize it. The common clauses, in their typical order of appearance, are:

SELECT,
FROM,
WHERE,
GROUP BY,
HAVING,
ORDER BY.

SELECT Clause

The **SELECT** clause is used to specify the columns you want to retrieve from the database. You can select all columns using an asterisk (*) or specify individual column names.

Syntax: **SELECT** column1, column2, ...

FROM table_name;

or **SELECT** *

FROM table_name;

Example:

Consider a table named **Employees** with columns **EmployeeID**, **FirstName**, **LastName**, and **Salary**.

To retrieve the first name and last name of all employees:

```
SELECT FirstName, LastName
```

FROM Employees;

To retrieve all columns for all employees:SELECT *

FROM Employees;

FROM Clause

The **FROM** clause specifies the table(s) from which the data will be retrieved. This is a mandatory clause in almost all **SELECT** statements.

Syntax:SELECT column1, column2, ...

FROM table_name;

Example:

To retrieve employee IDs and salaries from the **Employees** table:SELECT EmployeeID,
Salary

FROM Employees;

WHERE Clause

The **WHERE** clause is used to filter records based on specified conditions. It extracts only those records that fulfill the given criteria.

Syntax:

SELECT column1, column2, ...

FROM table_name

WHERE condition;

Example:

To retrieve the first name and last name of employees who earn more than 50000:
SELECT FirstName, LastName

FROM Employees

WHERE Salary > 50000;

To retrieve all details of employees whose first name is 'John':SELECT *

FROM Employees

WHERE FirstName = 'John';

GROUP BY Clause

The **GROUP BY** clause groups rows that have the same values in specified columns into summary rows. It is often used with aggregate functions (e.g., **COUNT**, **MAX**, **MIN**, **SUM**, **AVG**).

Syntax:

SELECT column1, aggregate_function(column2)

FROM table_name

GROUP BY column1;

Example:

Consider a table named **Orders** with columns **OrderID**, **CustomerID**, and **Amount**.

To count the number of orders for each customer:

SELECT CustomerID, COUNT(OrderID) AS NumberOfOrders

FROM Orders

GROUP BY CustomerID;

HAVING Clause

The **HAVING** clause is used to filter groups based on specified conditions. It is similar to the **WHERE** clause but operates on the results of the **GROUP BY** clause. You cannot use aggregate functions directly in the **WHERE** clause to filter grouped results.

Syntax:SELECT column1, aggregate_function(column2)

FROM table_name

GROUP BY column1

HAVING condition;

Example:

To find customers who have placed more than 2 orders:

```
SELECT CustomerID, COUNT(OrderID) AS NumberOfOrders
```

```
FROM Orders
```

```
GROUP BY CustomerID
```

```
HAVING COUNT(OrderID) > 2;
```

ORDER BY Clause

The **ORDER BY** clause is used to sort the result set in ascending (**ASC**) or descending (**DESC**) order based on one or more columns. **ASC** is the default sort order.

Syntax: SELECT column1, column2, ...

```
FROM table_name
```

```
ORDER BY column1 ASC|DESC, column2 ASC|DESC;
```

Example:

To retrieve the first name, last name, and salary of all employees, sorted by salary in descending order:

```
SELECT FirstName, LastName, Salary
```

```
FROM Employees
```

```
ORDER BY Salary DESC;
```

To retrieve employee details, sorted by last name in ascending order, and then by first name in ascending order:

```
SELECT *
```

```
FROM Employees
```

```
ORDER BY LastName ASC, FirstName ASC;
```

Complete Query Structure Example

Putting all the clauses together, a more complex query might look like this:

Scenario: Find the average salary for each department where the average salary is greater than 60000, and display the results sorted by average salary in descending order.

Consider a table named `Employees` with columns `EmployeeID`, `FirstName`, `LastName`, `Department`, and `Salary`.

```
SELECT Department, AVG(Salary) AS AverageSalary
```

```
FROM Employees
```

```
WHERE Department IS NOT NULL
```

```
GROUP BY Department
```

```
HAVING AVG(Salary) > 60000
```

```
ORDER BY AverageSalary DESC;
```